RDFS

RDF 词表描述语言 1.1 · 规范速查

RDF SCHEMA 1.1 — OVERVIEW · VOCAB ON TOP OF RDF

01 总览 · 给 RDF 装上词表

PART 1 · OVERVIEW

RDFS = 给 RDF 的一袋句子装上词表:类有层级、属性有链、domain/range 定位置,推理规则把隐含的三元组免费拉出来。先看四层怎么咬合,再逐个拆零件。

RDF 底座	三元组成句 · 图是集合	模型不变
类词汇	Resource 为根 · subClassOf 树	定义而非校验
属性词汇	九个属性 · subPropertyOf 链	谓语词汇全在这
推理 · 实践	规则拉出闭包 · 五步造词表	机械规则 · 无 LLM

主线:词表即数据——类、属性、规则全是可查询的 RDF 三元组。

四层各管一摊:类词汇管分类,属性词汇管关系,推理规则管闭包,实践层管怎么用。

本卡结构 = 本图:概览 → 类与属性 → 推理 → 建模实践 → 边界与实操。

02 定位 · 给 RDF 的一袋句子装上词表

PART 1 · POSITION

RDF 只说「主谓宾」,不解释词与词的关系。RDFS 补上最小的一层语义:**类与子类、属性与子属性、定义域与值域、人读的注释**——够推理机干活,又不至于把人埋进逻辑。

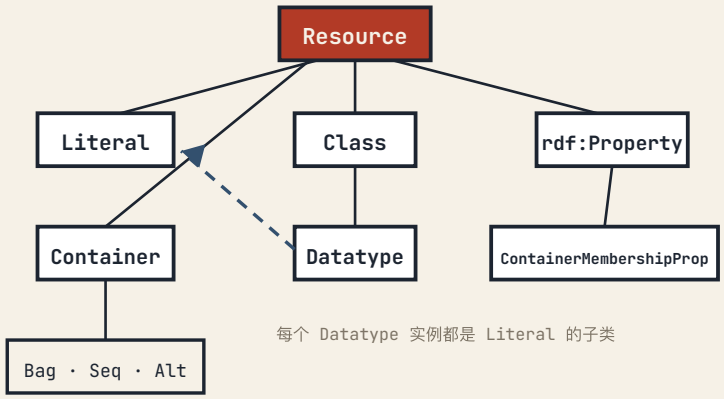
KEY POINT

RDF Schema 1.1,**W3C Recommendation 2014-02-25**,是 2004 年版的编辑性修订——技术内容原封未动,只为并入 RDF 1.1 文档族,标题从「RDF Vocabulary Description Language 1.0」改回朴素的名字。命名空间 `rdfs: = http://www.w3.org/2000/01/rdf-schema#`。

能力	词汇	一句话
分类	`rdfs:Class` + `rdf:type` + `rdfs:subClassOf`	声明类、挂实例、建类层级
属性层级	`rdf:Property` + `rdfs:subPropertyOf`	属性的「属于」链
用法声明	`rdfs:domain` / `rdfs:range`	这个属性的主语/宾语该是什么
注释	`rdfs:label` / `rdfs:comment` / `rdfs:seeAlso` / `rdfs:isDefinedBy`	给人看的说明书
容器	`rdfs:Container` / `rdfs:member`	开放成员的骨架(实体在 rdf:)

03 类的世界 · 一棵以 Resource 为根的树

PART 1 · THE CLASS TREE



每个 Datatype 实例都是 Literal 的子类

Class 自指合法:既是自己的实例也是自己的子类

一切皆 **Resource**;字面量也是资源(Literal ⊑ Resource),但它们只出现在宾语位置。类是资源的类(**Class**),而 Class 自己也是自己的实例——自指在这里是合法的。

虚线是 RDFS 最漂亮的机关:**每个数据类型都是 Literal 的子类**——xsd:integer、rdf:langString 全部挂在这条线上,类型系统由此统一。

没画进图里的

`rdfs:Datatype` 的双重身份(既是 Class 的实例又是其子类)、`rdfs:ContainerMembershipProperty` 的双重身份(⊑ rdf:Property,实例是 rdf:_n)——「类同时当自己的元层」是 RDFS 建模的常用技巧。

04 属性词汇 · 全部九个

PART 2 · PROPERTY VOCAB

属性	domain → range	语义
rdfs:subClassOf	Class → Class	传递;类层级的主干,可多继承,环 = 同一类
rdfs:subPropertyOf	Property → Property	传递;子属性出现处,超属性结论随之成立
rdfs:domain	Property → Class	用此属性的主语 ∈ 该类;**多条声明取交集**
rdfs:range	Property → Class	用此属性的宾语 ∈ 该类;**多条声明取交集**
rdfs:member	Resource → Resource	一切容器成员属性的祖先
rdfs:label	Resource → Literal	人读名;配语言标签做多语言
rdfs:comment	Resource → Literal	人读描述;同样可多语言
rdfs:seeAlso	Resource → Resource	「可能还有更多信息」,宽松;不要求可取回
rdfs:isDefinedBy	Resource → Resource	「定义它的资源」(词表本体);seeAlso 的子属性

05 domain / range 的真实语义 · 定义而非校验

PART 2 · DOMAIN & RANGE

规范语义

P rdfs:domain C 意思是:凡用 P 做谓语的主语,**推出**它是 C 的实例。宾语同理走 range。多条 domain/range 声明——主语/宾语属于**所有**这些类(交集)。

它做什么 INFERENCE

- 看到 :Tom :chase :Jerry + domain(:chase)=Cat,就**补出** :Tom a :Cat
- 从没见过的事实也能推:数据不全时反而有用
- 违反声明不报错——只会推出「它也是这个类」

它不做什么 NOT VALIDATION

- 不是完整性约束,写了错误数据不会失败
- 给不出「属性只在某类局部有此限制」——那是 OWL 的 restriction
- 想校验,请去 SHACL(PL.07);想表达复杂约束,去 OWL(PL.04)

经典误用

「我给 :hasWings 声明 domain :Bird,这样填错的人会报错」——不会。错误数据安静地通过,还多出一个「错误的主语 ∈ Bird」的推断。domain/range 的正确用法是**声明你希望推理机知道的事实**,不是设卡。

06 类速查表 · 一行一类

PART 2 · CLASS QUICK REF

类	是什么	备注
rdfs:Resource	一切资源的根类	所有其他类都是它的子类
rdfs:Class	「类」的类	自己的实例也是自己的子类
rdfs:Literal	字面量类(串、数…)	Resource 的子类;只出现在宾语位
rdfs:Datatype	「数据类型」的类	既是 Class 的实例又是其子类;每个实例 ⊑ Literal
rdfs:Container	开放容器的抽象父类	Bag / Seq / Alt 都 ⊑ 它
rdfs:ContainerMembershipProperty	rdf:_n 的类	⊑ rdf:Property;每个实例都是 member 的子属性
rdfs:Datatype 一族	rdf:langString / rdf:HTML / rdf:XMLLiteral	后两个非规范(引 PL.01 §13)

列表在哪

链表词汇 rdf:List / first / rest / nil 挂在 rdf: 命名空间(引 PL.01),RDFS 只负责容器的抽象父类。集合封闭性靠 rdf:List,不归 rdfs: 管。

07 推理规则 · RDFS 闭包从哪来

规则	形式	效果
子类传递	A ⊑ B ∧ B ⊑ C ⟹ A ⊑ C	层级逐级上溯
子类实例	x a A ∧ A ⊑ B ⟹ x a B	实例沿类链上浮
子属性传递	p ⊑ q ∧ q ⊑ r ⟹ p ⊑ r	属性链上溯
子属性结论	x p y ∧ p ⊑ q ⟹ x q y	用子属性即得超属性事实
domain 推型	x p y ∧ p domain C ⟹ x a C	主语获得类型
range 推型	x p y ∧ p range C ⟹ y a C	宾语获得类型
成员归并	C rdf:_n x ⟹ C rdfs:member x	容器成员归到 member

```
# 你写的
:Cat  rdfs:subClassOf :Animal .
:chase rdfs:domain :Cat ; rdfs:range :Mouse .
:Tom a :Cat ; :chase :Jerry .

# 推理机补出的(RDFS 蕴含)
:Tom a :Animal .     # 实例沿子类链上浮
:Tom a :Cat .        # domain 推型(这里恰好已写)
:Jerry a :Mouse .    # range 推型
```

官方规则 · rdfs1 - rdfs13(RDF 1.1 Semantics §9.2.1)

规则	前提	结论
rdfs1	aaa ∈ D(已识别的类型 IRI)	aaa rdf:type rdfs:Datatype
rdfs2	aaa rdfs:domain xxx · yyy aaa zzz	yyy rdf:type xxx
rdfs3	aaa rdfs:range xxx · yyy aaa zzz	zzz rdf:type xxx
rdfs4a / 4b	xxx aaa yyy	主语/宾语 rdf:type rdfs:Resource
rdfs5 / rdfs11	subPropertyOf / subClassOf 两段链	传递闭包一跳补全
rdfs6 / rdfs10	xxx rdf:type rdf:Property / rdfs:Class	自反:xxx ⊑ xxx
rdfs7	aaa ⊑ bbb · xxx aaa yyy	xxx bbb yyy
rdfs8	xxx rdf:type rdfs:Class	xxx rdfs:subClassOf rdfs:Resource
rdfs9	xxx ⊑ yyy · zzz rdf:type xxx	zzz rdf:type yyy
rdfs12	xxx rdf:type ContainerMembershipProperty	xxx rdfs:subPropertyOf rdfs:member
rdfs13	xxx rdf:type rdfs:Datatype	xxx rdfs:subClassOf rdfs:Literal

细则与完备性

规则应用穷尽即得 RDFS 闭包,多项式复杂度可判。两点深水区:①普通 RDF 语法下规则集**不完备**(字面量不能当主语),换「广义 RDF」才完备——工程实现都走广义;②数据类型(D-entailment)没有统一语法判据,`"25.0"^^xsd:decimal` 与 `"25"^^xsd:integer` 同值这类事要靠类型自身的词法映射。

08 建模惯用法 · 词表的自描述

PART 4 · MODELING IDIOMS

```
# 一个自描述词表的骨架:每个词汇带 label / comment / isDefinedBy
:Drug a rdfs:Class ;
    rdfs:label "药物"@zh-CN, "Drug"@en ;        # 多语言:每种语言一条
    rdfs:comment "用于预防、治疗、诊断的物质。"@zh-CN ;
    rdfs:subClassOf :MedicinalProduct .

:hasIndication a rdf:Property ;
    rdfs:label "有适应症"@zh-CN ;
    rdfs:domain :Drug ;
    rdfs:range :Disease ;
    rdfs:isDefinedBy <http://example.org/vocab/> .  # 指回词表本体
```

label vs comment 用法分界

- label:一个名字,短,图例/界面直接显示
- comment:一段解释,长,悬停提示/文档页用
- 两者都吃语言标签,别只写默认语言

seeAlso vs isDefinedBy 宽与严

- seeAlso:随便指,相关材料皆可
- isDefinedBy:指「定义我的词表文档」,是它的子属性
- 词汇术语一律挂 isDefinedBy,引用资料用 seeAlso

09 容器与具体化 · RDFS 提供的模型支架

PART 4 · CONTAINERS & REIF.

容器家族 open containers

- Bag 无序 / Seq 有序 / Alt 备选,形式上同构,全凭约定
- 成员资格:rdf:_1, _2…,都是 rdfs:member 的子属性
- 开放性:任何人可补 _n;要封闭请用 rdf:List

具体化 reification · 非规范

- rdf:Statement + subject/predicate/object 四件套,把「这句话」变成资源
- 同一个三元组可以有多个 Statement 实例——具体化不提供同一性
- 1.1 里这一节已标为非规范;做溯源请走 PROV-O(PL.11)

10 词表设计工作流 · 五步一份词表

PART 4 · VOCAB WORKFLOW

01 **起名**:词表 IRI 用自己的域名定死,路径语义化(…/vocab/1.0/),定了就别改——IRI 是承诺,改名即断链。

02 **排类层级**:subClassOf 从宽到窄;层深尽量不超过四,多继承可以有,环不可以有。

03 **挂属性**:每条属性给恰好一个 domain 和一个 range——多条声明取交集,想清楚了再写第二个。

04 **注释齐全**:label + comment 双语起步,isDefinedBy 指回词表;这三样是词表的「说明书页」。

05 **发布**:词表 IRI 内容协商返回 HTML(给人)与 Turtle(给机器);改版走新 IRI 或声明版本关系,别原地覆盖。

11 站在别人的词表上 · 常用外部词汇

PART 4 · EXTERNAL VOCABS

rdfs 本文 | SKOS 概念体系 · PL.03 | DCTERMS 文献元数据 · 频率/许可/出处 | FOAF 人与组织 | OWL-Time 时间区间/时刻 | PROV-O 溯源 · PL.11 | schema.org 网页语义/SEO

复用优先

先找现成词表,找不到再自建;自建的类用 `rdfs:seeAlso` 挂向近义的外部类,别急着 `owl:equivalentClass`——等价承诺是要推理机兑现的。

12 RDFS 不做什么 · 能力边界清单

PART 5 · CAPABILITY BOUNDARY

想要的能力	RDFS 给吗	去哪找
类与属性层级、domain/range	给	本体(本文)
数据校验、报错	不给	SHACL(PL.07)
类局部的属性限制(x 是 A 时其 p 值…)	不给	OWL restriction(PL.04)
基数、互斥、存在量词	不给	OWL 2(PL.04)
键、函数式属性	不给	OWL 2
属性的传递/对称声明	不给	OWL 2(仅 subClassOf/subPropertyOf 自带传递)

规范自己的话

「RDF Schema 提供描述机制,但不说应用该怎么用」——它是词表语言,不是规则引擎,更不是校验器。这份克制正是它能被所有推理机支持的原因。

13 实操坑清单

PART 5 · PITFALLS

01 domain/range 是**推理燃料不是护栏**:填错数据不报错,反而推出荒唐类型——声明前问自己「我真的想推出这个吗」。

02 多条 domain/range 取**交集**:想表达「或」,要么造一个并类,要么上 OWL 的 unionOf。

03 subClassOf / subPropertyOf **传递**:层级画环了,环上全体划等号——建模后先跑一次环检测。

04 查询容器别只写 rdfs:member:rdf:_n 才是实际写的谓词,member 是它们的归并。

05 label/comment 一词一语言一条:只有默认语言的词表,换语言界面就得返工。

06 词汇术语挂 isDefinedBy(精确),随手引用才用 seeAlso(宽松)。

07 字面量只做宾语:RDF 语法不允许字面量当主语或谓语,分类对象请给 IRI。

08 RDFS 推理是开放世界的:没有 domain 声明 ≠ 没有 domain;查「为什么推出这个」要看闭包。

09 想校验结构,别拿 domain/range 硬上:SHACL 的 PropertyShape 才是干这个的。

10 具体化四件套是老古董:非规范、无同一性、推理不联通——新项目一律 PROV-O。

14 工具速览 · RDFS 推理从哪进

PART 5 · TOOLING

工具	路线
Apache Jena	InfModel / 规则推理机,RDFS 闭包开箱即用
rdflib (Python)	内置 RDFS 闭包插件,脚本化批量推理
GraphDB / Virtuoso	图库内置 RDFS entailment 档位,加载时选档
Protégé + 推理机	HermiT / Pellet 走 OWL 路线,向下兼容 RDFS 层